Marine Race Arena: A Configurable HoloOcean Benchmark for Underwater Gate Racing and Team-Level Fleet Evaluation

Andrea Bedei, Lorenzo Bacchiani, Giovanni Pau and Roberto Girau
Department of Computer Science and Engineering, University of Bologna, Italy

***Abstract*—Autonomous underwater racing lacks a common framework for defining race scenarios, integrating controllers and comparing results consistently. We present Marine Race Arena, a configurable benchmark whose race model, controller contract and independent referee are separated from simulator-specific and vehicle-specific behavior through a replaceable adapter. Users can create tracks and configure gates, currents, obstacles, sensor profiles and scoring without modifying the core infrastructure. The benchmark first supports single-vehicle racing, then extends the same evaluation contract to staggered cooperative fleets with team scoring, inter-vehicle proximity monitoring and optional acoustic communication. Controllers receive only documented onboard observations, while privileged simulator state remains outside the controller boundary and is used for physical synthesis, validation, scoring and structured logging. The reference implementation provides three official clean tracks of different lengths, baseline controllers and a deterministic fallback adapter for tests. Reported HoloOcean experiments use a BlueROV2-class vehicle and include a matched comparison of two camera-assisted rule controllers: continuous visual servoing and center-then-commit passage. They establish clean-track feasibility and expose performance degradation under configured currents. Marine Race Arena therefore provides an extensible, reproducible foundation for individual and cooperative underwater racing research.**

***Index Terms*—underwater robotics, autonomous racing, HoloOcean, BlueROV2, simulation benchmark, referee, multi-robot systems, team-level evaluation**

I. Introduction

Autonomous racing has emerged as an effective research format for evaluating perception, planning, control and real-time decision-making within a single quantitative task under repeatable conditions [1]. In ground robotics, the F1TENTH platform provides standardized vehicles, circuits, metrics and baselines for continuous control and reinforcement learning [2]. At full scale, competitions such as the Abu Dhabi Autonomous Racing League (A2RL) push planning and control toward the limits of vehicle handling in head-to-head racing scenarios [3]. In aerial robotics, gate-based benchmarks and competitions have driven progress from modular perception and control pipelines [4] to learning-based policies capable of champion-level drone racing [5]. Across these domains, the availability of a well-defined racing task, fixed course geometry, explicit scoring rules and machine-readable outcomes has made autonomy methods easier to compare, reproduce and improve.

Underwater robotics has not yet benefited from an equivalent benchmark for speed-oriented autonomous racing along predefined gate courses. Existing marine simulators provide increasingly mature support for underwater vehicles, sensors and environments, but they mainly target intervention, inspection, navigation or sensor realism rather than racing as an evaluation protocol. In particular, they do not provide a compact underwater counterpart to ground and aerial racing benchmarks: a task defined by ordered gates, timing, penalties, currents, obstacles, independent scoring and reproducible outcomes. This gap matters because underwater racing is not simply aerial or ground racing transferred to a different simulator. Underwater vehicles are slow, strongly damped and affected by persistent currents; global localization is typically unavailable; inertial and Doppler sensing may drift; optical perception is limited in range and degraded by turbidity; acoustic cues are noisy and low bandwidth; and physical contact with gates, obstacles or other vehicles can be difficult to interpret. These properties change both the information available to a controller and the way in which performance should be evaluated.

A central requirement for such a benchmark is therefore a strict separation between autonomy and privileged simulation services. A controller should be evaluated using only information that would plausibly be available onboard, such as local sensing, physically received acoustic packets and controller-local state. Privileged simulator state, including global pose, exact gate geometry, official course progress and the true current field, must remain outside the controller boundary. The benchmark may use it internally to construct the physical environment and referee inputs, but official progress is evaluator-only and none of this state flows back to autonomy. Without this boundary, performance can depend on information that would not be available to a real vehicle, making results difficult to interpret and unfair to compare.

This paper presents Marine Race Arena, a configurable benchmark layer for autonomous underwater gate racing. Its benchmark model, controller contract and referee are vehicle-independent; replaceable adapters isolate simulator-specific sensing and actuation. The reference implementation and all reported experiments use HoloOcean [6] with a BlueROV2-class vehicle [7]. Rather than replacing a simulator or proposing an autonomy algorithm, Marine Race Arena defines an evaluation contract on top of a simulation backend. A JSON configuration specifies the world, ordered gates, sensors, cur-

rents, obstacles and scoring rules. Controllers receive only an official observation, while an independent referee uses privileged state to validate crossings and violations, assign penalties, manage timeouts and compute official scores.

The core task first evaluates one autonomous vehicle whose controller is integrated through a small interface (Section V). Fleet mode then extends the same mechanism to a cooperative team in which each vehicle runs its own controller and retains independent progress. The separation from evaluation lets the benchmark compare arbitrary controllers, including future learning-based ones, under the same official observation.

The contributions of this paper are as follows.

1) We present Marine Race Arena, an end-to-end configurable benchmark for autonomous underwater gate racing. Users can define complete race scenarios, including the environment, vehicle configuration, sensor profile, ordered gates, currents, obstacles, timing and scoring through a single configuration file. The release includes three ready-to-use official tracks and supports the creation of custom courses and experimental scenarios.
2) We provide an extensible autonomy interface for controller development and comparison. The framework includes two camera-assisted rule-based reference controllers and allows users to integrate their own controller through a minimal three-method interface without modifying the race runner, simulator adapter or evaluation logic. A documented official observation contract ensures that every controller is evaluated using the same non-privileged information.
3) We introduce an independent automated referee that uses privileged simulator state only for evaluation. It validates ordered gate crossings, detects rule violations, applies penalties and timeouts, ranks participants and produces structured event logs and machine-readable race summaries.
4) We support both single-vehicle racing and cooperative multi-vehicle evaluation within the same framework. Each vehicle maintains independent control and a locally estimated course state, while the referee maintains separate official scoring state for the team result. The fleet layer provides staggered or simultaneous starts, inter-vehicle proximity diagnostics, optional acoustic communication and distributed leader-follower coordination.
5) We provide a reproducible experimental validation in HoloOcean covering the three official tracks, comparison between the two reference controllers, disturbance-current scenarios and multi-vehicle coordination. The accompanying configurations, scripts and structured outputs allow the reported experiments to be reproduced and extended with new tracks, vehicles and control strategies.

The remainder of the paper is organized as follows. Section II reviews underwater simulators, autonomous-racing benchmarks and multi-AUV underwater cooperation. Section III formalizes the benchmark model and runtime architecture. Section IV describes the vehicle, the HoloOcean adapter, the sensor suite, the current model and the official observation contract. Section V specifies the controller interface and the rule-based baseline. Section VI formalizes gate validation, the referee state machine and single-rover scoring. Section VII describes staggered fleet execution and team scoring. Section VIII reports the experimental evaluation. Section IX outlines future work. Section X concludes the paper.

II. Related Work

We review related work along three axes: the underwater simulators that provide the physical and sensing substrate; the autonomous-racing benchmarks that define the evaluation format we adapt; and the multi-AUV cooperation and acoustic-communication literature that motivates and bounds the fleet mode. Table I summarizes the positioning of Marine Race Arena against representative systems.

A. Underwater Robotics Simulators

Several simulators provide marine vehicles and sensors. The UUV Simulator offers a Gazebo-based package for underwater intervention and multi-robot scenarios [8]. Stonefish provides custom marine-robotics physics and a wide range of underwater sensors behind a ROS interface [9]. DAVE builds on Gazebo to support acoustic and optical sensing for autonomous underwater vehicles [10]. HoloOcean uses Unreal Engine to render underwater scenes and exposes common marine sensors, sonar, RGB camera, IMU, DVL and depth, for one or more agents [6] and is the simulation backend we build on. These systems are complementary to Marine Race Arena: they simulate vehicles and sensors, whereas Marine Race Arena adds the race task, the official observation contract, the referee, the scoring and the output protocol on top of one of them. None of them ships a gate-racing benchmark with an impartial referee and consequently none enforces an information boundary between the controller and ground-truth state.

B. Autonomous Racing Benchmarks and Competitions

Racing benchmarks are valuable precisely because they expose end-to-end behavior under repeatable, adversarial constraints, as catalogued by the survey of Betz et al. [1]. F1TENTH provides a scaled autonomous car, standardized circuits and an evaluation environment for continuous control and reinforcement learning [2]; it is the closest in spirit to Marine Race Arena, but it targets ground vehicles with planar LiDAR and treats referee logic as only a partial, in-framework concern rather than a separated, first-class contract. At full scale, the A2RL challenge frames racing as multi-agent interaction at the limits of vehicle handling [3], while the A2RL league also runs a drone-racing format [12]. In aerial robotics, AlphaPilot established gate sequences as a demanding benchmark for fast perception and control [4] and Swift later demonstrated champion-level drone racing using deep reinforcement learning trained against a simulated opponent [5]. CARLA is not a racing benchmark, but it

TABLE I
COMPARISON WITH REPRESENTATIVE UNDERWATER SIMULATORS AND RACING BENCHMARKS.

System	Domain	Physics / sensing	Racing task	Multi-robot	Indep. referee	Official obs. contract
UUV Simulator [8]	Underwater	Gazebo, hydrodynamics	no	✓	no	no
Stonefish [9]	Underwater	Custom physics, marine sensors, ROS	no	no	no	no
DAVE [10]	Underwater	Gazebo, acoustic and optical sensing	no	✓	no	no
F1TENTH [2]	Ground	Scaled car, LiDAR, RL and control	✓	partial	partial	✓
AlphaPilot / Swift [4], [5]	Aerial	Quadrotor, vision-based gate racing	✓	no	partial	✓
CARLA [11]	Ground	Unreal Engine, urban sensing, tasks	partial	no	partial	✓
Marine Race Arena (this work)	Underwater	HoloOcean, BlueROV2, acoustic+camera	✓	✓	✓	✓

"Partial" marks a supported feature that is not a primary design contract.

illustrates how an Unreal-based ecosystem can standardize maps, sensors and tasks for driving research [11]. More broadly, standardized environment interfaces such as OpenAI Gym [13] and GPU-parallel robot-learning simulators such as Isaac Gym [14] show how a documented observation-action contract enables systematic comparison of learning-based controllers. Marine Race Arena borrows the gate-racing idea and the documented observation contract from these efforts, but adapts the benchmark to underwater sensing, slow and damped vehicle response, acoustic guidance, current disturbances and a referee whose scoring is computed entirely outside the control loop.

C. Multi-AUV Cooperation and Underwater Communication

Fleet operation underwater is constrained by the physics of the acoustic channel and by uncertain localization. Underwater acoustic links suffer from low bandwidth, long propagation delay, multipath and time-varying attenuation [15], so the assumption of continuous high-bandwidth, low-latency inter-vehicle communication that underpins many terrestrial multi-robot methods does not transfer. Surveys of AUV formation control document how these communication and navigation constraints shape the achievable coordination: Yang et al. analyze formation performance, control and communication capability jointly [16] and Yan et al. review formation-control methods and their dependence on relative-state estimation [17]. Consistent with this literature, Marine Race Arena does not assume shared ground-truth state or high-rate communication between vehicles. In v0.1 the fleet mode is deliberately conservative: it validates multiple HoloOcean agents, staggered releases, independent per-rover progress and team-level scoring.

D. Positioning

In contrast to the systems above, Marine Race Arena is the only one that simultaneously provides an underwater simulation backend, a gate-racing protocol with timing, an independent referee that uses privileged state, multi-rover and team-level scoring and an enforced official-observation contract that excludes ground-truth pose from control (Table I). It does not compete with the underwater simulators on physical fidelity, nor with the racing platforms on autonomy performance; it complements both by supplying the missing evaluation contract for underwater gate racing.

III. BENCHMARK MODEL AND SYSTEM ARCHITECTURE

We specify Marine Race Arena as an *evaluation problem* rather than a controller-design problem: the benchmark fixes the world, the task and the scoring and a participant supplies only a policy. This section defines the benchmark instance, the participant interface and the runtime architecture that enforces the separation between the two.

A. Benchmark Instance

Definition 1 (Benchmark instance). A Marine Race Arena instance is the tuple

$$\mathcal{E} = (\mathcal{W}, \mathcal{G}, \mathcal{S}, \mathcal{F}, \mathcal{C}, \mathcal{O}, \mathcal{P}, \mathcal{R}), \tag{1}$$

where:

- $\mathcal{W}$ is the *world*: an axis-aligned bounds box $\mathcal{B} = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}] \subset \mathbb{R}^3$ (with z positive-up, so the navigable volume satisfies $z < 0$) and the simulation environment selected through the adapter;
- $\mathcal{G} = (g_1, \ldots, g_M)$ is the *ordered* gate sequence; each gate is the tuple $g_k = (c_k,\, n_k,\, w_k,\, h_k)$ of center $c_k \in \mathbb{R}^3$, passage-direction vector $n_k \in \mathbb{R}^3$ (nonzero, not necessarily unit length) and inner aperture width w_k and height h_k;
- $\mathcal{S}$ is the start configuration: a spawn pose and an optional release delay $\tau_i \geq 0$ per participant;
- $\mathcal{F}$ is the finish condition: completion of the last gate of the last lap (Section VI);
- $\mathcal{C}$ is the current field $\mathbf{v}_c : \mathbb{R}^3 \times \mathbb{R}_{\geq 0} \to \mathbb{R}^3$ (Section IV);
- $\mathcal{O}$ is the obstacle set;
- $\mathcal{P} = \{1, \ldots, N\}$ is the participant set, each with a vehicle, sensor profile, controller, spawn and release delay;
- $\mathcal{R}$ is the referee configuration: gate-validation parameters, penalty values and scoring rules.

The gate basis is right-handed and derived from the passage direction alone, so the orientation of a gate is unambiguous and independent of any visual rotation (Section VI).

B. Participant Interface

Definition 2 (Policy). A participant i implements a policy

$$\pi_i : o_{i,t} \mapsto u_{i,t}, \tag{2}$$

TABLE II
DESIGN REQUIREMENTS FOR THE V0.1 BENCHMARK LAYER.

ID	Requirement
R1	Race configuration is declarative and reproducible from a single set of track JSON files.
R2	Official controller observations exclude ground-truth pose, simulator-only state and referee-derived course progress.
R3	Referee logic is independent of the participant controller and cannot be influenced by it.
R4	Gate dimensions, referee margins and track geometry are not changed to hide failures.
R5	Every run emits structured event logs and machine-readable summaries.
R6	Single-rover behavior is unchanged when fleet mode is disabled.
R7	Fleet mode aggregates a team score while preserving per-rover diagnostics.

mapping the official observation $o_{i,t}$ (Section IV) to a normalized, high-level, body-frame command

$$u_{i,t} = \left[u^{\text{surge}},\, u^{\text{sway}},\, u^{\text{heave}},\, u^{\text{yaw}}\right]^{\top} \in [-1, 1]^4. \quad (3)$$

The action of (3) is the portable, constrained interface that an adapter maps to the selected vehicle. In fleet mode the policy may return $(u_{i,t}, m_{i,t})$, where $m_{i,t}$ is an optional controller-authored communication payload and only $u_{i,t}$ is a physical action. The reference HoloOcean adapter also accepts a direct eight-thruster `thrusters` action for its BlueROV2-class agent. Time is discretized at a fixed control timestep Δt, with $t_k = k\,\Delta t$. The adapter clamps $u_{i,t}$ to actuator limits and applies depth-bound safety before mapping it to the vehicle; in official mode the policy reads only the official observation $o_{i,t}$, whose contract excludes privileged state (Section IV).

C. Design Requirements

The release-candidate design follows the seven requirements in Table II. They encode the central commitment that scoring logic is independent of the participant and cannot be manipulated through privileged feedback, together with the secondary commitment that enabling fleet mode must not silently change single-rover behavior.

D. Runtime Architecture

Fig. 1 shows the runtime architecture and the modules that realize it. The configuration loader parses and validates the track JSON into a typed configuration; the arena builder instantiates gates, independent beacons, the current field, obstacles and bounds. The race runner drives the discrete-time loop. On the simulation side, the adapter handles reset, actions, ticking and sensor extraction. An engine-free adapter implements the same interface for unit and plumbing tests but is not used for reported results. On the control side, the runner assembles the official observation from participant-local time, filtered onboard sensors and physically received packets, then passes it to the controller, which returns a command. On the scoring side, the referee consumes privileged simulator state, validates progress and emits structured events and summaries. The observation builder has no referee argument and no referee result flows back across the dashed boundary in Fig. 1; privileged state may support arena synthesis, adapter physics and evaluation, but remains outside the controller boundary. A controller may therefore disagree with the referee. The official trajectory is scored, while the disagreement is logged and audited offline rather than corrected by the runner.

IV. VEHICLE, SIMULATOR AND SENSING

This section describes the simulation backend, the vehicle and its sensors, the acoustic and current models, the declarative track configuration and the official observation contract enforced independently of the referee.

A. Simulator and Vehicle

The benchmark architecture is not tied to a particular vehicle: adapters provide simulator state, filtered sensing and action mapping to the common race and referee layers. The reference adapter, and every HoloOcean experiment reported here, uses HoloOcean [6] with a BlueROV2-class agent [7] in direct eight-thruster mode (HoloOcean `control_scheme` 0). The simulator advances at 30 physics ticks per second. The benchmark uses a control timestep of $\Delta t = 0.033\,\text{s}$, so each control step corresponds to one physics tick and the effective control rate is $\approx 30\,\text{Hz}$.

In the reference HoloOcean adapter, a high-level command $u = [u^{\text{surge}}, u^{\text{sway}}, u^{\text{heave}}, u^{\text{yaw}}]^{\top}$ is mapped to the eight BlueROV2 thrusters by a fixed mixing matrix. The four vertical thrusters receive u^{heave}; the four horizontal thrusters receive

$$\tau_{1\text{–}4} = s_\tau \begin{bmatrix} u^{\text{surge}} + u^{\text{sway}} + \kappa\, u^{\text{yaw}} \\ u^{\text{surge}} - u^{\text{sway}} - \kappa\, u^{\text{yaw}} \\ u^{\text{surge}} + u^{\text{sway}} - \kappa\, u^{\text{yaw}} \\ u^{\text{surge}} - u^{\text{sway}} + \kappa\, u^{\text{yaw}} \end{bmatrix}, \quad (4)$$

with yaw-mixing gain $\kappa = 0.35$ and thruster scale s_τ, each component clamped to the thruster limit. A controller may instead return the eight thruster values directly; the high-level interface is the default because it is simpler and vehicle-agnostic (a controller expresses intended body-frame motion while the framework handles the mixing and depth-safety), whereas direct thruster control is available when finer actuator authority is wanted.

B. Sensor Suite

Table III lists the onboard sensors. The official profile combines a forward RGB camera, a depth sensor, an IMU and a DVL; an onboard contact flag is also permitted when configured. World-frame velocity and ground-truth pose, location, rotation and dynamics sensors exist internally for simulation and referee evaluation, but are rejected at the adapter boundary and cannot appear in the official controller observation (Section IV-F).

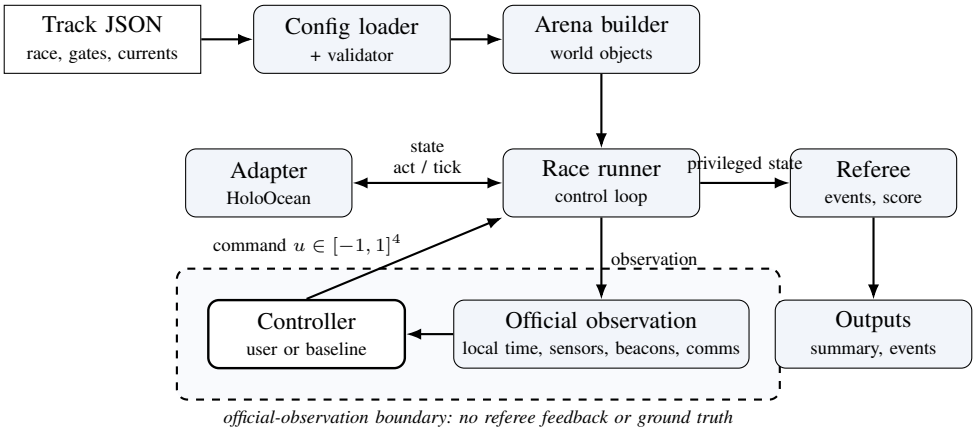

Fig. 1. Runtime architecture and official-observation boundary between the controller and referee.

TABLE III
ONBOARD SENSOR SUITE OF THE BLUEROV2.

Sensor	Rate	Output and configuration
Depth	30 Hz	Scalar depth (m); noise $\sigma = 0$.
IMU	30 Hz	Linear acceleration and angular rate, with bias terms.
DVL	15 Hz	Body-frame velocity (m/s); beam elevation $22.5°$, max range 50 m.
Front camera	30 Hz	RGB image, 640×480, $90°$ field of view.
Collision	optional	Boolean onboard contact flag when configured.

World-frame velocity, pose-derived heading/depth and environmental-current telemetry are not controller-visible in official mode. Pose and velocity sensors may exist internally for simulation and referee evaluation only.

C. Acoustic Beacon Model

Let $(\hat{n}, \hat{r}_g, \hat{s}_g)$ be the right-handed gate frame of (14), with $\hat{n}$ the passage normal and $\hat{s}_g$ the gate-up axis. Each gate carries one beacon, placed at $b_g = c_g + \delta_n \hat{n} + \delta_r \hat{r}_g + \delta_s \hat{s}_g$ relative to the gate center c_g (default offset $(0, 0, 0.35)$ m along $\hat{s}_g$). For a receiver at position p_r with heading ψ, let $\boldsymbol{\delta} = b_g - p_r$, let $\rho = \|\boldsymbol{\delta}\|$ be the geometric range and let $\tilde{\rho}$ be its noisy delivered measurement. The beacon reports $\tilde{\rho}$, a yaw-relative bearing, an elevation and a signal strength:

$$\rho = \|\boldsymbol{\delta}\|, \qquad \tilde{\rho} = \max(0, \rho + \eta_\rho), \tag{5}$$

$$\beta = \mathrm{wrap}\big(\operatorname{atan2}(\delta_y, \delta_x) - \psi\big), \tag{6}$$

$$\varepsilon = \operatorname{atan2}\big(\delta_z, \sqrt{\delta_x^2 + \delta_y^2}\big), \tag{7}$$

$$\mathrm{SS} = \max\Big(0, 1 - \tfrac{\tilde{\rho}}{\rho_{\max}}\Big),$$

where $\rho_{\max}$ is the beacon range and $\mathrm{wrap}(\cdot)$ maps to $(-180°, 180°]$. A transmission is not received beyond $\rho_{\max}$ or when a Bernoulli dropout fires. In the noisy acoustic mode, independent Gaussian noise $\mathcal{N}(0, \sigma_b)$ is added to β, ε and ρ; the official tracks use $\sigma_b \in \{0.2, 0.45, 0.6\}$ and dropout probabilities up to 0.04. Every ordered gate has one independent periodic transmitter with a unique contiguous identifier $\mathrm{B01}, \ldots, \mathrm{B}M$. All transmit regardless of referee state; at each update a rover receives every in-range, non-dropped packet. The framework neither activates a target transmitter nor labels a packet as the correct target. Packet randomness is seeded by the run seed, transmitter, receiver and transmission index, making reception independent of participant count and call order.

D. Current Field Model

The current field is a sum of analytic primitives evaluated at the vehicle position p and time t,

$$\mathbf{v}_c(p, t) = \sum_j \mathbf{v}_j(p, t), \tag{8}$$

where each primitive is one of the following. A *constant* term contributes a fixed velocity $\mathbf{v}_0$. A *localized jet* centered at c_j with radius R_j contributes, for $\|p - c_j\| \le R_j$,

$$\mathbf{v}_j = \mathbf{v}_0\, \omega(d), \qquad \omega(d) = \begin{cases} \max\big(0, 1 - \frac{d}{R_j}\big) & \text{linear,} \\ \exp\big(-\frac{d^2}{2\sigma^2}\big) & \text{Gaussian,} \end{cases} \tag{9}$$

with $d = \|p - c_j\|$ and $\sigma = \max(R_j/2, 10^{-6})$ and zero outside R_j. A *sinusoidal* term modulates one axis, $v_{j,\mathrm{axis}}(t) = v_{0,\mathrm{axis}} + A \sin(2\pi f t + \phi)$. A *vortex* of radius R_j, tangential speed V_t and vertical speed V_z contributes, with $r = \sqrt{\Delta x^2 + \Delta y^2}$ and tangent $\hat{\theta} = (-\Delta y, \Delta x)/r$,

$$\mathbf{v}_j = \big(\hat{\theta}_x\, V_t\, \omega(r),\ \hat{\theta}_y\, V_t\, \omega(r),\ V_z\, \omega(r)\big). \tag{10}$$

The provided `strong` profile superposes a constant set-flow of $[0.75, 1.05, 0]$ m/s, two Gaussian jets, a vortex and a vertical sinusoid; the `medium` profile scales these magnitudes by 0.5. Both named profiles are defined relative to the gate layout and are available on each official circuit. The HoloOcean adapter evaluates the field at every rover before each control step—one physics tick in the reported setup—and applies it through `env.set_ocean_currents`. Reported current experiments are rejected unless the run metadata confirms this physical coupling on the real HoloOcean adapter with no fallback.

E. Declarative Track Configuration

A track is a self-contained JSON file. Its principal sections are `race` (name, timing mode, laps and duration), `benchmark_task`, `world` (bounds and environment), `start` and `finish`, `track` and `gates` (the ordered gate sequence with centers, passage directions and aperture sizes), `beacon`, `currents`/`current_profiles`, `obstacle_generation`/`obstacles`, `participants` (vehicle, spawn, sensors, controller and optional `start_delay_s`) and `referee` (gate validation, penalties and scoring). The named `benchmark_task` (`clean_gate`, `obstacle_gate`, `current_gate` or `multi_rov`) selects a validation contract. Listing 1 shows a representative excerpt.

Listing 1. Representative excerpt of a track configuration.

```
"track": {
  "gate_inner_size_m": [1.5, 1.5],
  "gate_sequence": ["G01", "G02", "..."]
},
"gates": [
  { "id": "G01", "position": [4.0, -2.0, -3.5],
    "rotation_rpy_deg": [0.0, 0.0, 0.0],
    "passage_direction": [1.0, 0.0, 0.0] }
],
"beacon": {
  "range_m": 90.0, "noise_std": 0.2,
  "dropout_probability": 0.0,
  "update_rate_hz": 10.0
},
"participants": [
  { "id": "bluerov2_01", "vehicle": "BlueROV2",
    "controller": "rule_gate_baseline",
    "start_delay_s": 0.0 }
],
"referee": {
  "penalties": { "minor_collision_s": 5.0 },
  "gate_validation": {
    "vehicle_clearance_margin_m": 0.1 }
}
```

The three official v0.1 tracks are summarized in Table IV; all use $1.5\,\mathrm{m} \times 1.5\,\mathrm{m}$ apertures (requirement R4).

The `obstacle_gate` task accepts explicit fixed obstacles or deterministic random boxes generated between consecutive gates with a configured density and minimum clearance. The HoloOcean adapter records the requested and physically spawned counts, and an obstacle construction check is accepted only when they match. This verifies that obstacles can be instantiated on a circuit; it is deliberately separate from any claim that a reference controller avoids them.

F. Official Observation Contract

The controller `step` function receives a dictionary with the fields in Table V. The builder is structurally independent of the referee: its signature contains the track configuration, arena, adapter and participant state, but no referee object. It constructs participant-local elapsed time, asks the adapter only for allow-listed onboard sensors and asks the beacon field which periodic transmissions were physically received. Pose, location, rotation, dynamics, world-frame velocity, pose-derived heading or depth, environment-current vectors, exact gate geometry and all referee progress fields are forbidden. Numeric arrays are made JSON-safe while image arrays are preserved. A controller must infer disturbance effects from its onboard sensors, particularly body-frame DVL measurements, rather than read the current directly. When the optional inter-rover acoustic channel is enabled (Section VII), the observation additionally carries a `comms` inbox whose payloads are authored from another controller's own legal information.

Each beacon packet contains exactly `beacon_id`, `bearing_deg`, `elevation_deg`, `range_m`, `signal_strength` and the receiver-local `received_at_s`. No validity reason, gate id, expected-target flag, position or noise parameter is exposed. Silence is simply absence of a packet; the controller cannot distinguish range loss, dropout and an interval with no new transmission.

V. Controller Interface and Rule Baseline

A central usability goal of Marine Race Arena is that a custom autonomy stack is integrated by implementing a small interface, without touching the runner, the referee or the adapter. This section specifies that interface, the dynamic loading mechanism, the action mapping and the two camera-assisted rule controllers used in the reported single-rover comparison.

A. Controller Interface

A controller implements three methods: `reset(mission_info)`, called once before the run with a minimal static assignment; `step(observation)`, called every control tick with the official observation of (2) and returning the command of (3); and `close()`, called at teardown. The reset dictionary contains exactly `participant_id`, `initial_beacon_id`, `total_beacons`, `laps` and per-axis `command_limits`. Fleet missions additionally contain the configured release order, release index and predecessor id. They contain no referee target, progress, status, timing, world, current, obstacle, adapter or benchmark-task metadata. The contract is duck-typed: any object exposing the three callables is accepted, so a participant need not depend on framework base classes. A manual controller may raise a dedicated stop exception from `step` to end a run cleanly; other exceptions are recorded as controller errors. Listing 2 shows a minimal locally progressing example.

Listing 2. Minimal custom controller.

```
class MyController:
   def reset(self, mission_info):
      self.tracker = LocalCourseTracker(
         initial_beacon_id=mission_info[
            "initial_beacon_id"],
         total_beacons=mission_info[
            "total_beacons"],
         laps=mission_info["laps"])

   def step(self, observation):
      sensors = observation["sensors"]
      state = self.tracker.update(
         local_time_s=observation["local_time_s"],
         beacons=observation["beacons"],
```

TABLE IV
OFFICIAL V0.1 TRACK CONFIGURATIONS.

Track file	Track name	Gates	Laps	Total gates	Length (m)	Intended use
`marine_race_horseshoe_bay.json`	Marine Race Horseshoe Bay	12	1	12	93.8	Clean-gate horseshoe baseline
`marine_race_vertical_serpent.json`	Marine Race Vertical Serpent	17	1	17	118.3	Vertical and slalom gate sequencing
`marine_race_mixed_endurance.json`	Marine Race Mixed Endurance	22	1	22	206.3	Longer endurance and current-oriented layout

TABLE V
TOP-LEVEL FIELDS OF THE OFFICIAL OBSERVATION.

Field	Meaning
`local_time_s`	Elapsed time since this rover's release; it is not official race time.
`sensors`	Filtered onboard dictionary containing only configured camera, depth, IMU, DVL and optional collision measurements.
`beacons`	List of physically received packets from independent transmitters; it may be empty or contain several ids.
`comms`	Present only when the optional inter-rover acoustic channel is enabled: an `inbox` of messages received from teammates, each with the sender id, the controller-authored payload and a receiver-local reception timestamp (Section VII).

TABLE VI
HIGH-LEVEL CONTROLLER COMMAND FIELDS OF (3).

Field	Interpretation
`surge`	Longitudinal body-frame thrust; positive moves forward.
`sway`	Lateral body-frame thrust; sign follows the adapter mixing of (4).
`heave`	Vertical thrust; additionally constrained by depth-safety logic near the bounds.
`yaw`	Yaw-rate command; sign follows the adapter mixing of (4).

```
        camera_image=sensors.get("FrontCamera"),
        dvl_velocity=sensors.get("DVLSensor"))
    if state.finished:
        return {"surge": 0.0, "sway": 0.0,
                "heave": 0.0, "yaw": 0.0}
    # ... map state.beacon and camera to a command ...
    return {"surge": 0.3, "sway": 0.0,
            "heave": 0.0, "yaw": 0.0}

def close(self):
    pass
```

The high-level command fields are listed in Table VI; values are normalized to $[-1, 1]$ and clamped by the framework before the action mapping. In fleet mode a controller may additionally return a small `message` payload alongside its command and read incoming teammate messages from the observation, using the optional inter-rover acoustic channel of Section VII.

B. Dynamic Loading

A controller is selected at run time in one of three ways: a built-in alias such as `rule_gate_baseline` or `rule_gate_center_then_commit`; a dotted module path with an explicit class; or a file path together with `--controller-class`, which is imported dynamically. For inspection and data collection, the runner can also be launched with the manual `keyboard` or `pygame` controllers. This lets an external policy be evaluated without modifying the package:

```
python -m marine_race_arena.scripts.run_marine_race \
  --track .../marine_race_horseshoe_bay.json \
  --benchmark-task clean_gate \
  --controller path/to/my_controller.py \
  --controller-class MyController \
  --adapter holoocean --official --headless
```

C. Action Mapping

The framework clamps the returned command to $[-1, 1]^4$ and applies a depth-safety rule that suppresses heave that would drive the vehicle past $z_{\min}$ or $z_{\max}$. The selected adapter then maps this portable body-frame command to its vehicle; the reference HoloOcean adapter uses (4). That adapter may alternatively receive raw eight-thruster values, while the high-level interface remains the vehicle-independent default.

D. Controller-Local Course Progression

Both official rule controllers own a `LocalCourseTracker`; the runner supplies no progression signal. The tracker starts at the public mission assignment `initial_beacon_id` and maintains a local expected beacon, completed count, lap and status through

$$\textsc{Search} \rightarrow \textsc{Approach} \rightarrow \textsc{VisualAlign} \rightarrow \textsc{Commit} \rightarrow \textsc{VerifyExit} \rightarrow \textsc{Advance},$$

ending in FINISHED after the final locally confirmed beacon. It filters the received list for its own expected id; packets from other transmitters are not labeled or selected by the framework. Temporary visual loss, a missing packet, an isolated range jump, a range rise without a prior close approach and a commit without forward DVL displacement cannot advance the state.

Entry into COMMIT normally requires four consecutive camera frames with a gate detection of confidence at least 0.42, normalized center error $|c_x| \leq 0.13$, $|c_y| \leq 0.32$ and either area fraction at least 0.03 or confidence at least 0.60, together with expected-beacon range $\rho \leq 3.2\,\mathrm{m}$ and bearing $|\beta| \leq 8^\circ$. A two-frame close-range branch handles a consistently detected but cropped aperture only for $\rho \leq 1.6\,\mathrm{m}$ and the same bearing bound. Exit confirmation then requires all of the

following onboard evidence: at least $1.2\,\mathrm{m}$ of forward displacement integrated from DVL; four close packets establishing a filtered minimum no larger than $1.30\,\mathrm{m}$; a subsequent $0.9\,\mathrm{m}$ range rise (or a persistent $0.35\,\mathrm{m}$ rear-sector turnaround after prolonged rear evidence); six camera-on frames in which the previously locked gate has disappeared; and two fresh packets with $|\beta| \geq 100^\circ$. Only then does the tracker increment exactly once. After each non-final local advancement it holds an exit-clearance phase for $2.5\,\mathrm{s}$; after the final advancement it enters FINISHED directly and commands zero. A failed commit or verification timeout returns to acquisition without changing the expected id. Controller-local snapshots and independent referee crossings are logged separately for offline comparison and never fed back into control.

E. Baseline Based on Rules

The reference controller is deliberately interpretable rather than optimal. Its role is to provide a deterministic policy that is exercised across the clean tracks and exposes its failures under difficult geometry, currents or dense fleet interaction. Obstacle construction is validated separately and no obstacle-avoidance capability is claimed. Let β and ρ be the beacon bearing and range, d the measured depth and $d^\star$ the nominal hold depth. The policy is organized into the four explicit components summarized in Fig. 2.

- *Bearing alignment and surge gating.* The controller aligns the rover before allowing sustained forward motion. Yaw tracks the beacon bearing through a clamped proportional law with a small deadband,

$$u^{\mathrm{yaw}} = \mathrm{clamp}\big(k_\psi\,\beta,\, -\bar{u}_{\mathrm{yaw}},\, \bar{u}_{\mathrm{yaw}}\big), \qquad \bar{u}_{\mathrm{yaw}} = 0.16, \tag{11}$$

  and surge is gated by alignment. Sustained forward motion is admitted only when $|\beta| < 24^\circ$; for larger errors the controller turns without forward drive, or applies a small reverse brake when it is within $3\,\mathrm{m}$ of the gate. When aligned, surge is capped at $\bar{u}_{\mathrm{surge}} = 0.46$ and scales linearly with range and bearing alignment.
- *Depth hold.* The controller records the initial measured depth $d^\star$ and uses it as a nominal hold depth while beacon elevation guides the approach to each gate. With measured depth d and vertical velocity v_z, the implemented depth-hold term is

$$u^{\mathrm{heave}}_{\mathrm{depth}} = \mathrm{clamp}\big(0.20(d - d^\star) - 0.08 v_z,\, -0.16,\, 0.16\big), \tag{12}$$

  which is blended with the beacon-elevation heave as $u^{\mathrm{heave}} = 0.70\,u^{\mathrm{heave}}_{\mathrm{beacon}} + 0.30\,u^{\mathrm{heave}}_{\mathrm{depth}}$. When a gate is visible, the vertical image error contributes an additional correction.
- *Visual centering.* This component refines the acoustic approach in the final meters. When the front camera detects a gate, normalized image errors $(c_x, c_y) \in [-1, 1]^2$ generate proportional sway, yaw and heave corrections. Forward passage remains gated by the acoustic bearing and by centered-image thresholds $(|c_x|, |c_y|) \leq (0.13, 0.30)$.

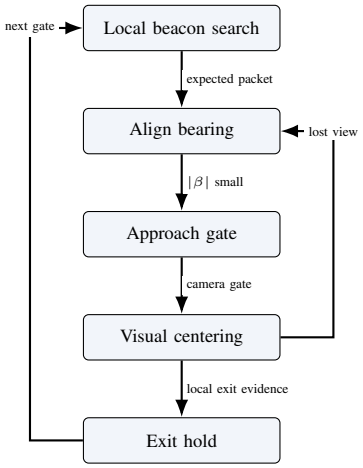

Fig. 2. Rule-based navigation around the controller-local course tracker.

- *Exit clearance and command smoothing.* After a non-final local passage confirmation, the controller clears the structure for $2.5\,\mathrm{s}$ with forward surge, holds the onboard measured passage depth and permits only a limited expected-beacon heading correction. Each confirmed gate establishes the depth reference for the next segment, which is required by the Vertical Serpent course. All commands are low-pass filtered, $u_t = (1-\alpha)\,u_{t-1} + \alpha\,u_t^{\mathrm{raw}}$ with $\alpha = 0.45$, and rate-limited per axis to avoid abrupt thrust changes.

F. A Center-Then-Commit Rule Controller

The second controller changes the gate-passage command rather than the observation, tracker or command envelope. It uses the same received beacon packets, front camera, depth and DVL feedback, command caps, smoothing and local confirmation logic as `rule_gate_baseline`. During COMMIT and VERIFYEXIT, the continuous-servo controller continues bounded lateral and vertical image correction while damping yaw from the onboard gyroscope. The center-then-commit controller instead stops chasing image-centroid changes: it sets image-driven sway to zero, holds the depth measured at lock, damps yaw from the IMU and permits only a small acoustic lateral correction while maintaining forward surge. Let q_t and a_t be the camera-target confidence and image-area fraction and let $(c_{x,t}, c_{y,t})$ be the normalized image errors. The normal lock condition is

$$\begin{aligned} \chi_t = \mathbb{1}\Big[&\rho_t \leq 3.2\,\mathrm{m},\ |\beta_t| \leq 8^\circ, \\ &q_t \geq 0.42,\ (a_t \geq 0.03\ \vee\ q_t \geq 0.60), \\ &|c_{x,t}| \leq 0.13,\ |c_{y,t}| \leq 0.32\Big], \\ \mathrm{commit}_t \iff &\sum_{k=0}^{3} \chi_{t-k} = 4. \end{aligned} \tag{13}$$

The same tracker decides when both controllers have passed; neither observes a valid-crossing event. A commit with no

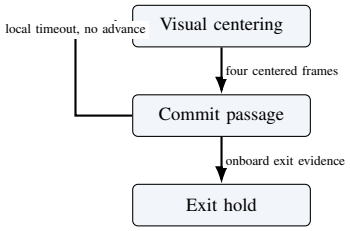

Fig. 3. Center-then-commit passage with controller-local confirmation and recovery.

sufficient motion times out after 14 s, and incomplete exit evidence times out after 16 s, in both cases returning to local acquisition without incrementing progress. Section VIII-E compares the two passage commands under matched HoloOcean conditions.

VI. Referee, Gate Validation and Scoring

The referee evaluates progress independently of the controller (requirement R3) using privileged ground-truth state. This section formalizes the gate-crossing test, the referee state machine and its arbitration order, the penalty model and the single-rover score.

A. Gate Frame and Crossing

Each gate's orientation derives solely from its passage direction n_g, giving a right-handed frame

$$\hat{n} = \frac{n_g}{\|n_g\|}, \quad \hat{r}_g = \frac{\hat{z} \times \hat{n}}{\|\hat{z} \times \hat{n}\|}, \quad \hat{s}_g = \hat{n} \times \hat{r}_g, \tag{14}$$

with world-up $\hat{z} = (0, 0, 1)$; if the passage direction is vertical ($\|\hat{z} \times \hat{n}\| \approx 0$), $\hat{r}_g$ falls back to the world $\hat{x}$ axis so the frame stays well defined. For consecutive referee-side positions p_{t-1}, p_t, the signed distances to the gate plane are

$$d_{t-1} = \hat{n}^\top (p_{t-1} - c_g), \qquad d_t = \hat{n}^\top (p_t - c_g). \tag{15}$$

A candidate crossing requires a sign change in the *correct direction*,

$$d_{t-1} \le 0 \le d_t \ \wedge \ (p_t - p_{t-1})^\top \hat{n} > 0, \tag{16}$$

i.e. travel from the negative to the positive side along the normal. The intersection of the segment with the plane is

$$\lambda = \frac{-d_{t-1}}{d_t - d_{t-1}}, \quad q = p_{t-1} + \lambda\,(p_t - p_{t-1}), \quad \lambda \in [0, 1], \tag{17}$$

and the crossing is accepted only when q lies inside the aperture, shrunk by the configured safety margin m_g,

$$\left|\hat{r}_g^\top (q - c_g)\right| \le \tfrac{w_g}{2} - m_g, \qquad \left|\hat{s}_g^\top (q - c_g)\right| \le \tfrac{h_g}{2} - m_g. \tag{18}$$

Gates must be passed *in sequence*: the referee tests only the expected gate g_ℓ, advancing the index $\ell \leftarrow \ell + 1$ on a valid crossing and incrementing the lap when the sequence completes. A crossing of a *non-target* gate aperture is a missed-gate event; a sign change in the wrong direction is logged but neither penalized nor counted as progress. Fig. 4 illustrates the test and Fig. 5 the principal automatic lifecycle.

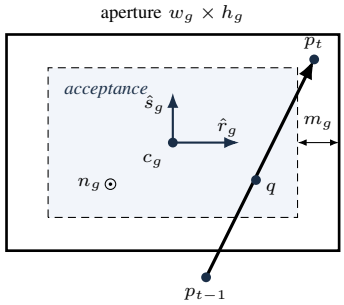

Fig. 4. Gate-crossing geometry and the safety-margin-adjusted acceptance region.

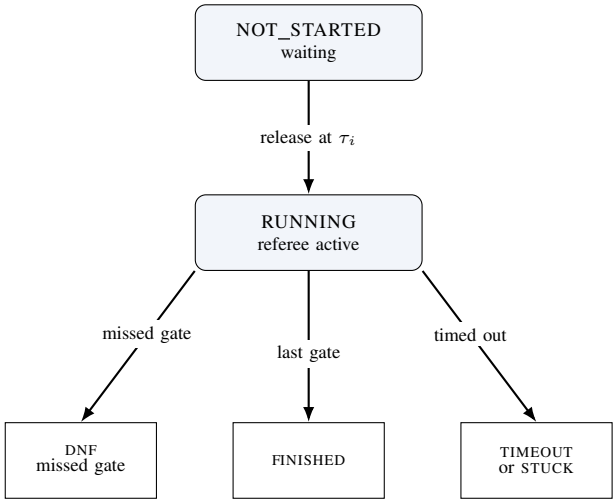

Fig. 5. Simplified automatic participant lifecycle; manual-stop and controller-error terminals are omitted.

B. Referee State Machine and Arbitration

Each participant occupies one of the states NOT_STARTED, RUNNING, FINISHED, DNF, TIMEOUT, STUCK, DSQ, MANUAL_STOP or CONTROLLER_ERROR; all but the first two are terminal. A participant is released to RUNNING when the race clock reaches its start delay (Section VII). On each tick of a running participant, events are evaluated in a fixed priority order that defines the arbitration: (1) a controller error transitions to CONTROLLER_ERROR and returns; (2) out-of-bounds; (3) generic collision; (4) obstacle collision; (5) stuck accumulation; (6) duration timeout; (7) gate progression. Steps (2) to (5) accrue penalties without terminating; only (1), (6), a missed gate in (7) or an external gate-timeout produce a terminal state. To avoid double counting, an obstacle-collision tick suppresses the generic collision flag. Repeated collision, obstacle and out-of-bounds charges use independent cooldowns; inter-vehicle proximity uses its own team-level cooldown, stuck is latched per episode and terminal events are not cooldown-limited.

C. Penalties and Termination

Penalties accumulate into a per-participant total P_i; the defaults applied in v0.1 are listed in Table VII. The non-trivial

TABLE VII
DEFAULT REFEREE EVENTS, PENALTIES AND TERMINAL CONDITIONS.

Event	Penalty	Terminal
Gate, bar or generic collision	5.0 s each	no
Obstacle collision	5.0 s each[†]	no
Out of bounds	10.0 s each	no
Stuck (soft)	15.0 s per episode	no
Missed gate	n/a	yes (DNF)
Timeout	n/a	yes
Gate-timeout stuck	n/a	yes
Inter-vehicle proximity event	0 (off/diag.); 5.0 s per pair (penalize)	no (team-level)

[†]Per-obstacle value if specified, else the collision default. Collision, obstacle, out-of-bounds and inter-vehicle proximity charges use independent configured cooldowns (default 1.0 s); stuck uses one latch per episode and terminal events have no cooldown.

conditions are formalized as follows. The instantaneous speed is $s_t = \|p_t - p_{t-1}\|/\Delta t$; a stuck accumulator integrates low-speed time,

$$T^{\text{stuck}} \leftarrow \begin{cases} T^{\text{stuck}} + \Delta t, & s_t < v_{\text{stuck}} \\ 0, & \text{otherwise,} \end{cases} \tag{19}$$

and a soft penalty is charged once per continuous low-speed episode when $T^{\text{stuck}} \geq T^{\text{stuck}}_{\max}$, with $v_{\text{stuck}} = 0.02\,\text{m/s}$. The configured $T^{\text{stuck}}_{\max}$ is $45\,\text{s}$ on Horseshoe Bay and $65\,\text{s}$ on Vertical Serpent and Mixed Endurance. An out-of-bounds event is $p_t \notin \mathcal{B}$. A duration timeout, disabled by default, fires when $t - t_{\text{green}} > T_{\max}$. A missed gate sets DNF when `missed_gate_dnf` is enabled (the default), but carries no time penalty, so a disqualifying error is never cheaper than a slow-but-legal lap.

D. Single-Rover Score and Ranking

For a finished participant the official time T_i^{official} is measured between the first and last gate (or green-to-finish, per the timing mode) and the score is the penalized time

$$T_i^{\text{pen}} = T_i^{\text{official}} + P_i. \tag{20}$$

Ranking places finished participants before unfinished ones and is defined by the ascending lexicographic key

$$\kappa_i = \begin{cases} (0, T_i^{\text{pen}}, 0, 0, 0, \text{id}_i), & f_i = 1, \\ (1, -G_i^{\text{done}}, n_i^{\text{coll}}, P_i, \text{dist}_i, \text{id}_i), & f_i = 0, \end{cases} \tag{21}$$

where $f_i = 1$ denotes a finished rover, G_i^{done} is the number of completed gates, n_i^{coll} the collision count and dist_i the distance to the next gate; the participant id is the deterministic final tie-break. Thus finished rovers sort by penalized time, while unfinished rovers sort by progress, then fewer collisions, then fewer penalties, then proximity to the next gate, so partial runs remain informative rather than being discarded.

E. Event Logging

Every run emits a line-delimited event log and a machine-readable summary (requirement R5). Events include `gate_passed`, `wrong_direction`, `collision`, `obstacle_collision`, `out_of_bounds`, `stuck`, `race_finish` and `dnf`, each timestamped and attributed to a participant. The summary records, per participant, the status, official and penalized times, completed gates and counts of collisions, out-of-bounds, missed gates and stuck events, together with the final ranking; in fleet mode it additionally carries the team summary of Section VII.

VII. FLEET AND TEAM EVALUATION

Fleet mode runs several rovers as a single cooperative team through the same circuit, so the benchmark can measure how quickly the team completes the course, rather than pitting them as competitors against each other (requirement R7). The benchmark models one team only: all rovers share a single simulated world, each controller keeps an independent local course estimate and the referee separately keeps one scoring state per rover. Enabling fleet mode does not alter single-rover behavior (R6).

A. Staggered Release and Spawn Geometry

Each participant i is assigned a release delay $\tau_i = (i-1)\,\Delta\tau$ for a start gap $\Delta\tau$, so rovers enter the course at $0, \Delta\tau, 2\Delta\tau, \ldots$ A waiting rover is sent a zero command, its controller `step` is *not* called and neither stuck nor gate-timeout timers accumulate; at release the referee logs a `participant_released` event and resets the rover's progress timers, so a delayed start is never misclassified as stuck or timed out. Spawn poses are laterally separated about the base spawn to reduce launch interference: using zero-based $k = 0, \ldots, N-1$, the k-th rover is offset by

$$o_k = (-1)^k \left\lceil \frac{k}{2} \right\rceil s \tag{22}$$

along the right axis $(-\sin\psi_0, \cos\psi_0)$ of the base yaw ψ_0, giving the alternating pattern $0, -s, +s, -2s, +2s, \ldots$ for spacing s; a candidate that would fall outside the world bounds $\mathcal{B}$ is scaled inward until admissible.

B. Inter-Vehicle Proximity Diagnostics

Inter-vehicle proximity events are detected on the referee side and are never exposed to controllers. For two running rovers a, b the detector applies a separable cylinder test

$$\sqrt{(x_a - x_b)^2 + (y_a - y_b)^2} \leq r_{xy} \;\wedge\; |z_a - z_b| \leq r_z, \tag{23}$$

with hysteresis (a pair is released only when the horizontal distance exceeds a release threshold r_{rel} or the vertical distance exceeds r_z) and a refractory cooldown to debounce sustained proximity. The v0.1 thresholds are $r_{xy} = 0.8\,\text{m}$, $r_z = 0.75\,\text{m}$, $r_{\text{rel}} = 1.05\,\text{m}$ and a $1.0\,\text{s}$ cooldown. Three modes are provided: `off` disables detection and is the command-line default; `diagnostic` logs inter-vehicle proximity events without changing any score; `penalize` also charges one team-level penalty per pair event. The fleet experiments use diagnostic mode explicitly.

C. Inter-Rover Acoustic Communication

Because the fleet is a single cooperative team, Marine Race Arena provides an optional channel so that rovers can share information while running the course. It is off by default and changes nothing for single-rover runs (R6). A controller transmits by returning a small `message` payload alongside its command and receives an `inbox` of teammates' messages in its observation; what a message contains and how it is used are entirely the user's responsibility, so the framework provides only the transport.

To stay faithful to the medium rather than model a perfect radio link, the channel reproduces the constraints of the underwater acoustic channel of Section II. A message broadcast by rover i reaches a teammate j only if the two are within a maximum range $R_{\max}$, after a range-dependent delay

$$\tau_{ij} = \frac{d_{ij}}{c} + \tau_{\text{proc}}, \tag{24}$$

where d_{ij} is the inter-rover distance, c the speed of sound and τ_{proc} a fixed processing delay; each delivery is additionally dropped with probability p_{loss}, payloads are capped to a small size to model the low bandwidth and a rover may not transmit faster than a configured per-sender interval. Packet loss is drawn from a seeded generator, so a run stays reproducible.

The channel preserves the official observation contract. A payload is authored by the sending controller, which sees only its official observation, so it can carry only information that controller may legally observe; the framework uses the true rover positions solely to compute the channel physics (d_{ij}, τ_{ij}, range and loss) and never injects pose, geometry or other privileged state into a delivered message. Inter-rover communication is therefore the substrate for the leader-follower coordination of Section VII-D.

D. Leader-Follower Coordination

A staggered team of identical controllers stays separated in time, but a heterogeneous team does not: a faster follower released behind a slower leader catches it and they bunch up at the gates, where every rover converges on the same aperture center. We therefore provide a coordination controller that keeps the team a spaced convoy using only legal information and wraps a gate-passing controller that exposes a `LocalCourseTracker`, rather than replacing it, so coordination and gate navigation stay separable. Both official camera-assisted controllers satisfy this interface.

The controller is a thin, fully distributed layer over the acoustic channel of (24). Each rover is assigned a predecessor from the static release order in its legal mission information; the frontmost rover has no predecessor and races freely. Within the channel's transmit-rate limit, every rover periodically broadcasts exactly the three fields `local_beacon_index`, `local_lap` and `local_status`, read from its own base controller's `LocalCourseTracker`. The receiver treats these fields as estimates, retains the most recently received predecessor report and ignores it after a 2.5 s freshness window. Let $\hat{g}_i$ denote cumulative locally estimated progress reconstructed from the reported lap and beacon index. A follower yields while its predecessor is fewer than Δg locally estimated gates ahead,

$$\text{yield}_j \iff \hat{g}_{\text{pred}(j)} - \hat{g}_j < \Delta g, \tag{25}$$

and otherwise it runs its wrapped base controller unchanged. While yielding, the wrapper still updates the base tracker from onboard observations but commands zero motion. This deliberate wait can therefore trigger the independent stuck penalty and remains visible in the result. A predecessor locally reported as finished, a missing report or a report older than the freshness window stops blocking. With the channel disabled no report is delivered and the wrapper reduces to the uncoordinated base behavior.

The design turns controller-local progress estimates into physical separation. Because the gates are ordered and spatially separated, a two-gate estimated lead ($\Delta g = 2$, the default) asks the follower to retain at least one intervening gate. We also evaluate $\Delta g = 1$ as an ablation in Section VIII. The policy reads only the official observation, its own tracker, the static predecessor assignment and controller-authored messages; referee progress can disagree with either local estimate but never changes a coordination decision.

E. Team-Level Score

Only the `team_summary` is the official fleet result; per-rover rows are diagnostics. For a team $\mathcal{P}$ of N rovers with G_{exp} expected gates each,

$$G^{\text{exp}}_{\text{team}} = N\, G_{\text{exp}}, \qquad G^{\text{done}}_{\text{team}} = \sum_{i \in \mathcal{P}} G^{\text{done}}_i. \tag{26}$$

The team finishes only if every rover finishes,

$$\text{finished}_{\text{team}} = \bigwedge_{i \in \mathcal{P}} \text{finished}_i, \tag{27}$$

in which case the team elapsed time spans the first release to the last finish,

$$T_{\text{team}} = \max_i T^{\text{finish}}_i - \min_i T^{\text{release}}_i, \tag{28}$$

and the penalized team score aggregates every per-rover penalty together with the team-level inter-vehicle term,

$$T^{\text{pen}}_{\text{team}} = T_{\text{team}} + \sum_{i \in \mathcal{P}} P_i + P_{\text{ivc}}, \tag{29}$$

where P_i is the per-rover penalty total of (20) and P_{ivc} counts each inter-vehicle proximity event once per pair, not once per rover. Fig. 6 illustrates the aggregation.

VIII. Experimental Evaluation

This evaluation does not seek an optimal underwater racing controller. It tests whether the benchmark, official observation contract, referee and team scoring behave as specified. It also reports where the reference baseline fails.

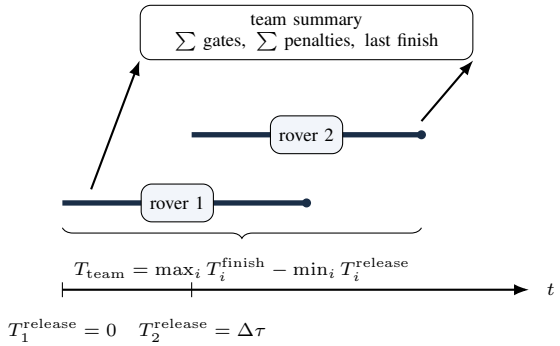

Fig. 6. Team-level scoring for a staggered two-rover fleet.

A. Protocol and Metrics

The final matrix contains 78 real-HoloOcean runs under one frozen source fingerprint. Clean single-rover validation crosses three tracks, two controllers and seeds 0–4 (30 runs). Current validation crosses the `medium` and `strong` Horseshoe Bay profiles, both controllers and the same five seeds (20 runs). Homogeneous two-rover fleet validation uses Horseshoe Bay, a 90 s start gap and both controller choices over five seeds (10 runs). Three-rover coordination uses clean Horseshoe Bay, start gaps 8.0 and 0.0 s, seeds 0–2 and matched coordinated/uncoordinated conditions (12 runs); a further six coordinated runs repeat both gaps with the yield margin reduced from two locally estimated gates to one.

The two official controllers are the continuous-servo `rule_gate_baseline` and `rule_gate_center_then_commit`. Coordination deliberately forms a heterogeneous convoy: the leader uses continuous servo and both followers use center-then-commit, while every rover maintains its own local tracker. All performance experiments use the reference HoloOcean adapter with a BlueROV2-class agent, $\Delta t = 0.033$ s, official onboard-only observations, motion compensation disabled, no obstacles and unmodified 1.5×1.5 m gate apertures (R4). Time limits are 560 s for Horseshoe Bay, current, fleet and coordination runs, 900 s for Vertical Serpent and 1300 s for Mixed Endurance. Fallback is disabled. Current runs are accepted only when metadata confirms physical `set_ocean_currents` coupling.

Separate setup checks, not controller-performance trials, instantiate both named current profiles and seeded static random obstacles on every official circuit. These checks confirm that the same gate geometry supports medium or strong current and physical obstacle placement; they do not claim that either reference gate follower avoids obstacles.

The reported metrics are course completion, completed gates, official time, penalized time from (20), gate/world collisions, out-of-bounds events and stuck events. Fleet and coordination validation additionally report inter-vehicle proximity events and team elapsed and penalized times. Completed-gate and event statistics include all seeds. Time statistics include finished runs only and are omitted when no seed finishes. Controller-local progress is compared with independent referee progress for every rover. False or missed local advancements, inconsistent final counts and inconsistent finish state fail the progress audit rather than being repaired or hidden; ordinary delay for a matched advancement is reported separately.

B. Single-Rover Clean Tracks

Tables IX and VIII report the clean-track results over five seeds. Continuous servo finishes every Horseshoe Bay and Vertical Serpent run but only $2/5$ Mixed Endurance runs. Center-then-commit finishes all Horseshoe Bay and Mixed Endurance runs and $4/5$ Vertical Serpent runs. The mean completed-gate counts are respectively 12.0, 17.0 and 17.0 for continuous servo and 12.0, 14.8 and 22.0 for center-then-commit. No clean run records a gate/world collision, out-of-bounds event or stuck event. The non-completions are therefore preserved rather than hidden by collision penalties or modified course geometry.

C. Staggered Fleet and Team Scoring

The bottom block of Table VIII reports homogeneous two-rover fleet validation on Horseshoe Bay over five seeds. With either controller, every team finishes $24/24$ gates without gate/world collisions, inter-vehicle proximity events, out-of-bounds events or stuck events. Mean team elapsed and penalized time are both 303.092 ± 2.874 s for continuous servo and 307.976 ± 5.227 s for center-then-commit. The 90 s release gap deliberately produces a low-contact case that validates multi-rover spawning, staggered release, independent per-rover referee state, ranking and team aggregation for both official controllers.

D. Behavior Under Currents

Performance degrades as current strength increases. Under the Horseshoe Bay `medium` profile, continuous servo finishes $2/5$ runs, averages $8.4/12$ completed gates and 43.4 gate/world collisions per run; its finished-only official and penalized times are 337.887 ± 13.563 s and 692.887 ± 53.563 s. Center-then-commit finishes $3/5$, averages $8.8/12$ gates and 25.2 gate/world collisions, with finished-only times of 217.261 ± 1.229 s and 223.928 ± 1.504 s. Neither controller finishes a `strong` run: continuous servo averages $3.0/12$ gates and no gate/world collision, while center-then-commit averages $3.2/12$ gates and 0.8 gate/world collisions. Metadata confirms physical current coupling in every run. These outcomes retain the unmodified gates, margins and current profiles (requirement R4); robust current rejection remains future work (Section IX).

E. Single-Rover Controller Comparison

This matched comparison uses the two camera-assisted implementations of Section V: continuous visual servoing in `rule_gate_baseline` and staged passage in `rule_gate_center_then_commit`. Each runs alone on clean Horseshoe Bay over seeds 0–4 with identical track, simulator, timing, observation contract and referee settings.

TABLE VIII
FINAL HOLOOCEAN CLEAN-TRACK, CURRENT AND HOMOGENEOUS TWO-ROVER FLEET RESULTS OVER FIVE SEEDS PER CASE.

Case	Ctrl	FIN	Gates	Raw (s)	Penal. (s)	GW	IV	OOB/S
Vertical clean	Cont.	5/5	17.0	236.947 ± 5.745	236.947 ± 5.745	0.0	0.0	0/0
	CTC	4/5	14.8	241.139 ± 1.567	241.139 ± 1.567	0.0	0.0	0/0
Mixed clean	Cont.	2/5	17.0	394.746 ± 2.772	394.746 ± 2.772	0.0	0.0	0/0
	CTC	5/5	22.0	410.659 ± 7.850	410.659 ± 7.850	0.0	0.0	0/0
Horseshoe medium	Cont.	2/5	8.4	337.887 ± 13.563	692.887 ± 53.563	43.4	0.0	0/0
	CTC	3/5	8.8	217.261 ± 1.229	223.928 ± 1.504	25.2	0.0	0/0
Horseshoe strong	Cont.	0/5	3.0	—	—	0.0	0.0	0/0
	CTC	0/5	3.2	—	—	0.8	0.0	0/0
Fleet, gap 90 s	Cont.	5/5	24.0	303.092 ± 2.874	303.092 ± 2.874	0.0	0.0	0/0
	CTC	5/5	24.0	307.976 ± 5.227	307.976 ± 5.227	0.0	0.0	0/0

Cont. denotes continuous servo and CTC center-then-commit. Times are mean±population standard deviation over finished runs only; Raw is official single-rover time and team elapsed time for fleet rows. Gates, gate/world collisions (GW) and inter-vehicle proximity events (IV) are means per run over all five seeds; fleet values are team-level. OOB/S reports out-of-bounds/stuck events.

TABLE IX
FINAL HOLOOCEAN SINGLE-ROVER RESULTS ON CLEAN HORSESHOE BAY OVER FIVE SEEDS. TIMES USE FINISHED RUNS ONLY; GW IS THE MEAN NUMBER OF GATE/WORLD COLLISIONS PER RUN.

Controller	FIN	Gates	Official (s)	Penal. (s)	GW
Continuous servo	5/5	12.0	194.456 ± 4.286	194.456 ± 4.286	0.0
Center-then-commit	5/5	12.0	197.683 ± 6.312	197.683 ± 6.312	0.0

Both controllers finish all 12 gates in every seed with no gate/world collision, out-of-bounds event or stuck event. Mean official and penalized time are 194.456 ± 4.286 s for continuous servo and 197.683 ± 6.312 s for center-then-commit. Thus the staged controller is 3.227 s (1.7%) slower on average in this matched case; the experiment does not support a speed or collision advantage for it on clean Horseshoe Bay. The comparison is limited to this track and the five evaluated seeds.

F. HoloOcean Diagnostic Validation

HoloOcean diagnostic validation tests local leader–follower coordination under contact dynamics. It uses clean Horseshoe Bay in official mode with three rovers, no currents or obstacles, two start gaps (8.0 and 0.0 s) and three seeds. The leader runs continuous servo and both followers run center-then-commit; every controller advances its own LocalCourseTracker, and heartbeats carry only local estimates. Inter-vehicle mode is diagnostic and no controller or course parameter is retuned between conditions.

With the primary two-gate yield margin, coordination finishes $3/3$ runs at both gaps and records no gate/world collision or inter-vehicle proximity event. At an 8 s gap, no coordination finishes only $2/3$, averages $34.333/36$ team gates, 127.333 gate/world collisions and 2.0 inter-vehicle proximity events per run; coordinated team time is 285.725 ± 3.129 s. At simultaneous release, both policies finish $3/3$. No coordination is faster in raw time (219.692 ± 2.027 s) but averages 9.333 gate/world collisions and 2.333 inter-vehicle proximity events, while coordination eliminates both event types but records one stuck event per run and 303.310 ± 1.076 s penalized team time. The one-gate-margin ablation finishes all six runs without gate/world collisions, inter-vehicle proximity events, out-of-bounds events or stuck events; its mean team times are 266.024 ± 6.765 s at gap 8.0 and 262.339 ± 6.287 s at gap 0.0. This evidence remains limited to one clean track, three rovers, two gaps and three seeds (Table X).

G. Reproducibility

Every run emits a structured event log and summary. The repository provides the exact commands used here [18]. A single configuration-driven entry point and per-track JSON files make each official run reproducible from one command. The single-rover, fleet and coordination sweeps are scripted, so the HoloOcean result tables can be regenerated end to end.

IX. FUTURE WORK

We see four concrete next steps.

(i) Cooperative fleet strategies beyond leader-follower convoying. The leader-follower coordination of Section VII-D has undergone HoloOcean diagnostic validation on clean Horseshoe Bay (Section VIII-F). The immediate next step is to extend that diagnostic validation to other tracks, currents, obstacles and larger fleets. Later work can examine formation keeping, cooperative overtaking, dynamic role assignment and robustness to uncertain relative localization or heavier acoustic loss. Such strategies must respect the low bandwidth and high latency of acoustic communication [15] described in the AUV-formation literature [16], [17].

(ii) Competitive multi-team and heterogeneous-rover evaluation. Extend the cooperative fleet setting to multiple rover teams or rover families with different dynamics and sensing suites. They would compete under the same referee and observation contract. This extension requires team-isolated scoring, ranking across teams, controlled interaction policies and normalization rules that compare heterogeneous platforms without hiding their physical differences.

(iii) Current compensation and broader disturbance validation. The three official circuits already expose the graded

TABLE X

FINAL HOLOOCEAN THREE-ROVER COORDINATION RESULTS ON CLEAN HORSESHOE BAY OVER THREE SEEDS PER CONDITION. THE LEADER USES CONTINUOUS SERVO, BOTH FOLLOWERS USE CENTER-THEN-COMMIT AND INTER-VEHICLE MODE IS DIAGNOSTIC.

Gap (s)	Policy	FIN	Gates	Team raw (s)	Team penal. (s)	GW	IV	OOB/S
0.0	No coord.	3/3	36.0	219.692 ± 2.027	266.359 ± 52.372	9.333	2.333	0.0/0.0
	LF(2)	3/3	36.0	288.310 ± 1.076	303.310 ± 1.076	0.0	0.0	0.0/1.0
	LF(1)	3/3	36.0	262.339 ± 6.287	262.339 ± 6.287	0.0	0.0	0.0/0.0
8.0	No coord.	2/3	34.333	254.034 ± 17.358	564.034 ± 327.358	127.333	2.0	0.0/0.0
	LF(2)	3/3	36.0	285.725 ± 3.129	285.725 ± 3.129	0.0	0.0	0.0/0.0
	LF(1)	3/3	36.0	266.024 ± 6.765	266.024 ± 6.765	0.0	0.0	0.0/0.0

FIN counts team runs in which all rovers finished. Times are team-level mean±population standard deviation over finished runs only. Gate/world collisions (GW), inter-vehicle proximity events (IV), and out-of-bounds/stuck events (OOB/S) are means per run over all three seeds. LF(2) is the primary two-gate-margin policy; LF(1) is the one-gate-margin ablation.

`medium` and `strong` profiles of (8)–(10); the present quantitative current sweep is limited to Horseshoe Bay. Future controllers can be evaluated on the remaining circuits and can explicitly reject these disturbances. One legal direction is a cascaded design with an inner loop that servos DVL-measured body velocity to the guidance setpoint, beneath outer beacon and vision guidance, without access to environmental-current telemetry.

(iv) Learning-based controllers. The official observation contract can serve directly as a reinforcement-learning interface. The state contains received beacon range, bearing and elevation; depth; IMU; body-frame DVL; and camera data. The normalized command of (3) is the action. A reward can combine locally estimated sequential progress with the time and collision penalties of (20), while official progress remains evaluator-only. Training and evaluation should retain the HoloOcean vehicle and contact dynamics used for the reported benchmark results. This direction follows work on champion-level drone racing against a simulated opponent [5] and robot-learning environments [13], [14] without weakening the official observation contract.

X. CONCLUSION

This paper presented Marine Race Arena, a configurable benchmark layer that isolates vehicle-specific and simulator-specific behavior behind a replaceable adapter; the reported implementation uses HoloOcean and a BlueROV2-class agent. It defines single-vehicle evaluation, then extends it to staggered cooperative fleets with team scoring. Controllers see only the official observation, while an independent referee uses privileged state for crossings, penalties, timeouts, ranking and scores. We formalized the benchmark instance, gate geometry, single-rover and team scoring and staggered fleet semantics, then grounded them in the implementation.

Across the five-seed clean validation, continuous servo finishes $5/5$ Horseshoe Bay, $5/5$ Vertical Serpent and $2/5$ Mixed Endurance runs; center-then-commit finishes $5/5$, $4/5$ and $5/5$ respectively. Both homogeneous two-rover fleet variants finish all five seeds without gate/world collisions, inter-vehicle proximity events, out-of-bounds events or stuck events. Under the `medium` current profile, continuous servo finishes $2/5$ and center-then-commit $3/5$; neither controller finishes a `strong` run. These failures remain benchmark evidence because gate geometry, referee margins and current profiles are unchanged.

In the matched clean Horseshoe Bay comparison, both camera-assisted controllers finish every seed without collision; continuous servo averages $194.456\,\mathrm{s}$ and center-then-commit $197.683\,\mathrm{s}$, so this case supports neither a speed nor a collision advantage for staged passage. In the three-rover diagnostic, the primary leader–follower policy finishes all six runs and eliminates gate/world collisions and inter-vehicle proximity events, although simultaneous release produces one stuck event per seed. Without coordination, one $8\,\mathrm{s}$-gap team fails and contact costs are substantial. The one-gate-margin ablation finishes all six runs cleanly and more quickly than the primary margin in this limited sample. Execution and artifact-contract checks pass with zero failures, but the overall onboard-only audit fails on 15 runs with controller-local/referee progression disagreement; these outcomes are preserved rather than concealed.

REFERENCES

[1] J. Betz, H. Zheng, A. Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, and R. Mangharam, "Autonomous vehicles on the edge: A survey on autonomous vehicle racing," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458–488, 2022.

[2] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning," in *Proceedings of the NeurIPS 2019 Competition and Demonstration Track*, ser. Proceedings of Machine Learning Research, vol. 123, 2020, pp. 77–89. [Online]. Available: https://proceedings.mlr.press/v123/o-kelly20a.html

[3] S. Hoffmann, S. Sagmeister, T. Betz, J. Bongard, S. Büttner, D. Ebner, D. Esser, G. Jank, S. Goblirsch, A. Langmann, M. Leitenstern, L. Ögretmen, P. Pitschi, A.-K. Schwehn, C. Schröder, M. Weinmann, F. Werner, B. Lohmann, J. Betz, and M. Lienkamp, "Head-to-head autonomous racing at the limits of handling in the A2RL challenge," *arXiv preprint arXiv:2602.08571*, 2026, abu Dhabi Autonomous Racing League; preprint, not peer reviewed. [Online]. Available: https://arxiv.org/abs/2602.08571

[4] P. Foehn, D. Brescianini, E. Kaufmann, T. Cieslewski, M. Gehrig, M. Muglikar, and D. Scaramuzza, "AlphaPilot: Autonomous drone racing," in *Robotics: Science and Systems (RSS)*, 2020. [Online]. Available: https://www.roboticsproceedings.org/rss16/p081.html

[5] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, 2023.

[6] E. Potokar, S. Ashford, M. Kaess, and J. G. Mangelson, "HoloOcean: An underwater robotics simulator," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 3040–3046. [Online]. Available: https://www.ri.cmu.edu/publications/holoocean-an-underwater-robotics-simulator/

[7] M. von Benzon, F. F. Sørensen, E. Uth, J. Jouffroy, J. Liniger, and S. Pedersen, "An open-source benchmark simulator: Control of a BlueROV2 underwater robot," *Journal of Marine Science and Engineering*, vol. 10, no. 12, p. 1898, 2022.

[8] M. M. M. Manhães, S. A. Scherer, M. Voss, L. R. Douat, and T. Rauschenbach, "UUV simulator: A Gazebo-based package for underwater intervention and multi-robot simulation," in *OCEANS 2016 MTS/IEEE Monterey*, 2016, pp. 1–8.

[9] P. Cieślak, "Stonefish: An advanced open-source simulation tool designed for marine robotics, with a ROS interface," in *OCEANS 2019 - Marseille*, 2019, pp. 1–6.

[10] M. M. Zhang, W.-S. Choi, J. Herman, D. Davis, C. Vogt, M. McCarrin, Y. Vijay, D. Dutia, W. Lew, S. Peters, and B. Bingham, "DAVE aquatic virtual environment: Toward a general underwater robotics simulator," in *Proceedings of the IEEE/OES Autonomous Underwater Vehicles Symposium (AUV)*, 2022, pp. 1–8. [Online]. Available: https://arxiv.org/abs/2209.02862

[11] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 78, 2017, pp. 1–16. [Online]. Available: https://proceedings.mlr.press/v78/dosovitskiy17a.html

[12] Autonomous Robotics Research Center, Technology Innovation Institute, "A2RL: Abu dhabi autonomous racing league," https://a2rl.io, 2024, autonomous car and drone racing championship; accessed 2026-06-27.

[13] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "OpenAI gym," *arXiv preprint arXiv:1606.01540*, 2016. [Online]. Available: https://arxiv.org/abs/1606.01540

[14] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, "Isaac gym: High performance GPU-based physics simulation for robot learning," in *Proceedings of the NeurIPS 2021 Track on Datasets and Benchmarks*, 2021. [Online]. Available: https://arxiv.org/abs/2108.10470

[15] M. Stojanovic and J. Preisig, "Underwater acoustic communication channels: Propagation models and statistical characterization," *IEEE Communications Magazine*, vol. 47, no. 1, pp. 84–89, 2009.

[16] Y. Yang, Y. Xiao, and T. Li, "A survey of autonomous underwater vehicle formation: Performance, formation control, and communication capability," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 2, pp. 815–841, 2021.

[17] T. Yan, Z. Xu, and S. A. Gadsden, "Formation control of multiple autonomous underwater vehicles: A review," *Intelligence & Robotics*, vol. 3, no. 1, pp. 1–22, 2023.

[18] A. Bedei, "HoloDroneCompetition: Marine race arena repository," https://github.com/AndreaBedei1/HoloDroneCompetition, 2026, v0.1 release candidate, accessed 2026-06-27.